Machine Coding: Auto-focusing Form with Validation

- **Problem:** Build a sign-up form (name, email, password) that manages focus for the user: focus the first field when the form appears, jump to the **first invalid field** on submit, focus the field a server error belongs to, show an error summary whose links focus each field, and return focus to the first field after a successful submit. Include a show/hide password toggle that doesn't submit the form, live re-validation after the first attempt, and accessible error messages.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Focus rules + validation rules
Use cases + architecture + state	5 min	Ref map, derived errors
Form + labels + mount focus	8 min	Working form
Validation + focus first invalid field	12 min	Core feature
Server error + focus, success + refocus, summary links	12 min	Full focus flow
Password toggle, a11y attributes, polish	8 min	Accessible
Keyboard-only test + wrap-up	10 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"Focus is a side effect, not state, so I use refs. Instead of one `useRef` per field, I keep a single map of refs keyed by field name. Then 'focus the first invalid field' is just: find the first name in field order that has an error, and call `.focus()` on its ref."*

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Fields: Full name (≥ 2 chars), Email (valid format), Password (≥ 8 chars, a letter and a number).
- On mount: focus the name field.
- On submit with errors: show the errors, an error summary at the top, and focus the **first** invalid field (in visual order).
- After the first submit attempt: errors update live as the user types.
- Server error (email already registered): show it under the email field and focus it.
- On success: show a message, reset the form, focus the first field again.
- Show/hide password toggle that never submits the form.
- Error summary links move focus to the matching field.

Non-functional requirements

Concern	What it means here
Performance	Validation is a few string checks; derived on render. No extra re-renders for focus (refs).
Scalability	Field order in one array (`FIELDS`); refs in one map; adding a field = one entry + one rule.
Error handling	Client rules, server field errors, double-submit guard, inputs `readOnly` (not disabled) while submitting.
Accessibility	Real `<label>` s, `aria-invalid` , `aria-describedby` linking errors, focus moved deliberately (not randomly), `role="status"` for success, toggle with `aria-pressed` + `aria-controls` , visible focus rings.
Reusability	Pure `validateField` / `validateAll` / `firstInvalid` ; the ref-map pattern works for any form.
Extensibility	More fields, async validators, multi-step forms.

Clarifying questions to ask the interviewer

1. Which field should get focus on load, and is auto-focus wanted at all (it can disorient screen-reader users)?
2. When should errors show: on blur, on submit, or live after the first submit?
3. On submit with several errors: focus the first invalid field, or an error summary at the top?
4. What are the exact validation rules? Is there a server check (email taken)?
5. After a successful submit: reset and refocus, or navigate away?
6. Should Enter move to the next field, or submit (the browser default)?

Assumptions we will build with

- Focus the name field on mount (it's the only purpose of the page).
- Validate on submit, then live; first invalid field gets focus; a summary is shown too, with links.
- Server errors map to a field and focus it.

- Enter submits (standard behaviour); Tab moves between fields.
- Success resets and refocuses the first field.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Cursor already in "Full name"
Submit everything empty	Summary "Please fix 3 errors"; errors under each field; focus on Full name
Name filled, submit	Focus jumps to Email (the first invalid one); summary lists 2 errors
Type a valid email after that	Its error and summary entry disappear live
Click a summary link	Focus moves to that field (no page jump, no URL hash)
Tab to "Show" + Enter	Password becomes visible, `aria-pressed="true"` ; form is **not** submitted
Valid form, submit	"Submitting..." (button disabled, inputs read-only)
Server: "already registered"	Error under Email, summary updated, focus on Email
Edit the email	Server error cleared immediately
Valid submit succeeds	"Account created!" (announced), fields cleared, focus back on Full name
Press Enter repeatedly while submitting	Only one request
Screen reader on an invalid field	"Email, invalid entry, Enter a valid email..."

3. Design the architecture

Module	Responsibility
`components/AutoFocusForm.jsx`	Form state, ref map, focus rules, submit flow, rendering fields/summary/toggle
`utils/validate.js`	`FIELDS` (order), `validateField` , `validateAll` , `firstInvalid`
`utils/fakeApi.js`	`registerUser` : mock server with a field error for `taken@example.com`
`App`	Page shell

```
App
└── <AutoFocusForm>
      values, submitted, serverErrors, status, showPassword   (state)
      fieldRefs = { name: <input>, email: <input>, password: <input> }   (ref map)
      clientErrors = submitted ? validateAll(values) : {}      (derived)
      ├── error summary (links → focusField)
      ├── field: label + input {...fieldProps(name)} + error <p id="name-error">
      ├── password row + toggle (type="button")
      └── submit + status (role="status")
utils/validate.js ← pure rules + field order
utils/fakeApi.js  ← mock POST
```

Start simple. First version: three refs and an `if/else` chain (like the original). Then replace it with the ref map + `firstInvalid` as soon as you notice the repetition, before adding the server error and summary.

4. Design the state / data model

State	Purpose
`values`	`{ name, email, password }`
`submitted`	Has the user tried to submit? (turns on live validation)
`serverErrors`	`{ email: "This email is already registered" }`
`status`	`idle | submitting | success`
`showPassword`	Toggle state
`fieldRefs` (ref)	`{ [fieldName]: HTMLInputElement }`

```
const fieldRefs = useRef({});
const register = (name) => (el) => { fieldRefs.current[name] = el; };   // callback ref
const focusField = (name) => fieldRefs.current[name]?.focus();

useEffect(() => { focusField("name"); }, []);                           // focus on mount

const clientErrors = submitted ? validateAll(values) : {};              // derived
const errorFor = (name) => clientErrors[name] || serverErrors[name] || "";
const invalidFields = FIELDS.filter((f) => errorFor(f));                // drives the summary
```

Senior note

- **Focus is not state:** moving focus is an imperative side effect on a DOM node, so it lives in refs and event handlers, never in `useState`.
- **Derived errors:** the original stored `errors` and cleared each one on change; that gets out of sync (e.g. errors don't reappear when you break a field again). Here client errors are always computed from `values`; only *server* errors are stored, because only the server knows them.
- **One source of field order (`FIELDS`):** both validation and "first invalid" use it, so focus always follows the visual order.

5. Design the data structures and contracts

Field order and rules:

```
export const FIELDS = ["name", "email", "password"];
validateField("email", "a@b") → "Enter a valid email, e.g. name@example.com"
validateAll(values)           → { name: "", email: "Enter a valid…", password: "Password is required" }
firstInvalid(errors)          → "email"   // first in FIELDS order with a message
```

Mock API contract:

```
registerUser({ name, email, password }) → Promise<{ ok: true }>
// rejects with Error("This email is already registered") + { field: "email" } for taken@example.com
```

Real version: `POST /api/register` → `201` or `409 { field: "email", message: "…" }`.

ARIA wiring for one field:

```
<label for="email">Email</label>
<input id="email" aria-invalid="true" aria-describedby="email-error">
<p id="email-error">Enter a valid email, e.g. name@example.com</p>
```

6. Trace the data flow

User fills only the name and presses Enter:

```
Enter in a field → form submit → handleSubmit
  ↓ preventDefault; not submitting → setSubmitted(true)
  ↓ errors = validateAll(values) → { name: "", email: "Email is required", password: "Password is
required" }
  ↓ firstInvalid(errors) → "email" → fieldRefs.current.email.focus()
  ↓ React re-renders: clientErrors now computed → error <p>s, aria-invalid, summary "Please fix 2
errors"
  ↓ Screen reader (focus on email): "Email, edit text, invalid entry, Email is required"
User types a valid email → setValues → re-render → email error + summary entry disappear live
```

7. Important technical challenges

Challenge 1: focusing the first invalid field without one ref per field

- *Problem:* The first version used `nameRef`, `emailRef`, `passwordRef` and an `if / else if` chain. With 10 fields this is 10 refs and a long chain that silently breaks when fields are reordered.
- *Solution:* A ref map via callback refs + one ordered list of fields.

```
const fieldRefs = useRef({});
const register = (name) => (el) => { fieldRefs.current[name] = el; };
export const firstInvalid = (errors) => FIELDS.find((f) => errors[f]);

const first = firstInvalid(validateAll(values));
if (first) { fieldRefs.current[first]?.focus(); return; }
```

Challenge 2: the toggle button that submits the form

- *Problem:* A `<button>` inside a `<form>` defaults to `type="submit"`. In the first version, clicking "show" **submitted the form** (and ran validation, moving focus away). The original also rendered errors in `<spa>` (a typo, an unknown element) and had no labels, only placeholders.

- *Solution:* `type="button"`, plus state exposed with `aria-pressed` and linked with `aria-controls`; proper `<label>`s and `<p>` error elements.

```
<button type="button" onClick={() => setShowPassword((s) => !s)}
        aria-pressed={showPassword} aria-controls="password">
  {showPassword ? "Hide" : "Show"}
</button>
```

Challenge 3: focusing after an async server error

- *Problem:* The server error arrives after `await`. If the inputs are `disabled` during submit, `.focus()` on a disabled input does nothing, so focus is lost (it falls to `<body>`). The error must also stay visible while client validation is clean.
- *Solution:* `readOnly` while submitting (still focusable), store server errors separately, focus the field from the error's `field`, and clear the server error when the user edits that field.

```
try { await registerUser(values); /* success */ }
catch (err) {
  setServerErrors({ [err.field]: err.message });
  setStatus("idle");
  focusField(err.field);
}
// in handleChange:
if (serverErrors[name]) setServerErrors((prev) => ({ ...prev, [name]: "" }));
```

Challenge 4: errors that are announced and easy to reach

- *Problem:* Red text alone isn't announced, colour isn't enough, and with many errors users must hunt for them. An `alert()` on success (as in the original) is blocking and jarring.
- *Solution:* `aria-invalid` + `aria-describedby` on each input (the error is read when the field gets focus), an error summary whose links call `focusField`, and a polite `role="status"` message on success.

```
<a href={`#${f}`} onClick={(e) => { e.preventDefault(); focusField(f); }}>
  {LABELS[f]}: {errorFor(f)}
</a>
<p className="status" role="status">{status === "success" ? "Account created!…" : ""}</p>
```

8. Implement in the correct order

#	Step	Working result
1	Form with `<label>`s, controlled inputs, `noValidate`	Typing works
2	Focus the first field on mount (ref + effect)	Auto-focus
3	`validateField` / `validateAll` + error `<p>`s on submit	Errors visible

#	Step	Working result
4	`FIELDS` order + `firstInvalid` + ref map → focus first invalid	Core feature
5	`submitted` flag → live re-validation	Errors clear as you fix
6	`aria-invalid` + `aria-describedby`	Errors announced
7	Password toggle with `type="button"`, `aria-pressed`	No accidental submit
8	Async `registerUser` + `status` + double-submit guard + `readOnly`	Realistic submit
9	Server field error → store, show, focus; clear on edit	Server feedback
10	Success: reset, `role="status"`, refocus the first field	Ready for the next entry
11	Error summary with focusing links	Easy to reach all errors
12	Node self-check for the rules and `firstInvalid`	Logic proven

9. Cover all states and edge cases

State / edge case	Handling
Initial	Empty form, focus on Full name, no errors
Typing before any submit	No errors shown
Submit with errors	Errors + summary; focus on the first invalid field in visual order
After the first submit	Live re-validation; summary count updates
Whitespace-only name	"Name is required" (trimmed)
Password with only letters / only numbers	"Needs at least one letter and one number"
Submitting	Button disabled + "Submitting..."; inputs read-only (still focusable); Enter ignored
Server field error	Shown + focused; cleared when that field changes
Success	Message announced, form cleared, `submitted` reset, focus on Full name, password hidden again
Toggle password	Never submits; focus stays on the toggle
Summary link	Focuses the field; no hash in the URL; no page jump
Field removed from the DOM	`fieldRefs.current[name]?.focus()` → no crash
Browser autofill	Handled through `onChange`; `autoComplete` hints set

10. Final structure

```
_14_auto_focusing_form_and_validation/
├── index.html
└── src/
    ├── main.jsx
```

```
    ├── index.css                  ← global reset
    ├── App.jsx                    ← page shell
    ├── App.css                    ← form, errors, summary, focus styles
    ├── utils/
    │   ├── validate.js            ← FIELDS, validateField, validateAll, firstInvalid
    │   └── fakeApi.js             ← mock registerUser with a server field error
    └── components/
        └── AutoFocusForm.jsx      ← state, ref map, focus rules, rendering
```

11. Full working code

src/utils/validate.js

```
// Field order = visual order. Used to find the FIRST invalid field to focus.
export const FIELDS = ["name", "email", "password"];

const EMAIL = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;

export function validateField(name, value) {
  const v = value.trim();
  switch (name) {
    case "name":
      if (!v) return "Name is required";
      return v.length >= 2 ? "" : "Name must be at least 2 characters";
    case "email":
      if (!v) return "Email is required";
      return EMAIL.test(v) ? "" : "Enter a valid email, e.g. name@example.com";
    case "password":
      if (!value) return "Password is required";
      if (value.length < 8) return "Password must be at least 8 characters";
      return /\d/.test(value) && /[A-Za-z]/.test(value) ? "" : "Password needs at least one letter and
one number";
    default:
      return "";
  }
}

export const validateAll = (values) => Object.fromEntries(FIELDS.map((f) => [f, validateField(f,
values[f])]));

export const firstInvalid = (errors) => FIELDS.find((f) => errors[f]);
```

src/utils/fakeApi.js

```
// shortcut: fake POST /api/register; swap for fetch() when a backend exists.
// "taken@example.com" simulates a server-side field error.
```

```
export async function registerUser({ email }) {
  await new Promise((resolve) => setTimeout(resolve, 700));
  if (email.trim().toLowerCase() === "taken@example.com") {
    throw Object.assign(new Error("This email is already registered"), { field: "email" });
  }
  return { ok: true };
}
```

src/components/AutoFocusForm.jsx

```
import { useEffect, useRef, useState } from "react";
import { FIELDS, firstInvalid, validateAll } from "../utils/validate";
import { registerUser } from "../utils/fakeApi";

const EMPTY = { name: "", email: "", password: "" };
const LABELS = { name: "Full name", email: "Email", password: "Password" };

const AutoFocusForm = () => {
  const [values, setValues] = useState(EMPTY);
  const [submitted, setSubmitted] = useState(false); // validate live only after the first submit
attempt
  const [serverErrors, setServerErrors] = useState({});
  const [status, setStatus] = useState("idle"); // idle | submitting | success
  const [showPassword, setShowPassword] = useState(false);

  // One ref map instead of one useRef per field: scales to any number of fields.
  const fieldRefs = useRef({});
  const register = (name) => (el) => {
    fieldRefs.current[name] = el;
  };
  const focusField = (name) => fieldRefs.current[name]?.focus();

  useEffect(() => {
    focusField("name"); // auto-focus the first field when the form appears
  }, []);

  // Derived errors: client rules (after the first submit) + errors returned by the server.
  const clientErrors = submitted ? validateAll(values) : {};
  const errorFor = (name) => clientErrors[name] || serverErrors[name] || "";
  const invalidFields = FIELDS.filter((f) => errorFor(f));
  const isSubmitting = status === "submitting";

  const handleChange = (e) => {
    const { name, value } = e.target;
    setValues((prev) => ({ ...prev, [name]: value }));
    if (serverErrors[name]) setServerErrors((prev) => ({ ...prev, [name]: "" }));
    if (status === "success") setStatus("idle");
```

```
};

const handleSubmit = async (e) => {
  e.preventDefault();
  if (isSubmitting) return;
  setSubmitted(true);

  const first = firstInvalid(validateAll(values));
  if (first) {
    focusField(first); // jump straight to the first problem
    return;
  }

  setStatus("submitting");
  try {
    await registerUser(values);
    setValues(EMPTY);
    setSubmitted(false);
    setShowPassword(false);
    setStatus("success");
    focusField("name"); // ready for the next entry
  } catch (err) {
    setServerErrors({ [err.field]: err.message });
    setStatus("idle");
    focusField(err.field);
  }
};

const fieldProps = (name) => {
  const error = errorFor(name);
  return {
    id: name,
    name,
    ref: register(name),
    value: values[name],
    onChange: handleChange,
    readOnly: isSubmitting, // not disabled: disabled inputs can't receive focus after a server error
    "aria-invalid": Boolean(error),
    "aria-describedby": error ? `${name}-error` : undefined,
  };
};

const errorText = (name) =>
  errorFor(name) && (
    <p id={`${name}-error`} className="field-error">
      {errorFor(name)}
    </p>
```

```
    );

  return (
    <form className="form" onSubmit={handleSubmit} noValidate>
      {submitted && invalidFields.length > 0 && (
        <div className="summary">
          <p>
            Please fix {invalidFields.length} {invalidFields.length === 1 ? "error" : "errors"}:
          </p>
          <ul>
            {invalidFields.map((f) => (
              <li key={f}>
                <a
                  href={`#${f}`}
                  onClick={(e) => {
                    e.preventDefault();
                    focusField(f);
                  }}
                >
                  {LABELS[f]}: {errorFor(f)}
                </a>
              </li>
            ))}
          </ul>
        </div>
      )}

      <div className="field">
        <label htmlFor="name">{LABELS.name}</label>
        <input type="text" autoComplete="name" {...fieldProps("name")} />
        {errorText("name")}
      </div>

      <div className="field">
        <label htmlFor="email">{LABELS.email}</label>
        <input type="email" autoComplete="email" inputMode="email" {...fieldProps("email")} />
        {errorText("email")}
      </div>

      <div className="field">
        <label htmlFor="password">{LABELS.password}</label>
        <div className="password-row">
          <input type={showPassword ? "text" : "password"} autoComplete="new-password"
{...fieldProps("password")} />
          {/* type="button": a plain <button> inside a form would SUBMIT the form */}
          <button
            type="button"
```

```
            className="toggle"
            onClick={() => setShowPassword((s) => !s)}
            aria-pressed={showPassword}
            aria-controls="password"
          >
            {showPassword ? "Hide" : "Show"}
          </button>
        </div>
        {errorText("password")}
      </div>

      <button type="submit" disabled={isSubmitting}>
        {isSubmitting ? "Submitting..." : "Create account"}
      </button>

      <p className="status" role="status">
        {status === "success" ? "Account created! The form is ready for another entry." : ""}
      </p>
    </form>
  );
};

export default AutoFocusForm;
```

src/App.jsx

```
import "./App.css";
import AutoFocusForm from "./components/AutoFocusForm";

function App() {
  return (
    <main className="container">
      <h1>Auto-focus Form & Validation</h1>
      <p className="hint">
        Try submitting empty, or use <code>taken@example.com</code> to see a server error.
      </p>
      <AutoFocusForm />
    </main>
  );
}

export default App;
```

src/App.css

```
body { background: #f3f4f6; font-family: system-ui, sans-serif; color: #111827; }
.container { max-width: 440px; margin: 40px auto; padding: 24px; background: white; border-radius: 12px;
}
h1 { font-size: 24px; margin-bottom: 4px; }
.hint { color: #6b7280; font-size: 14px; margin-bottom: 16px; }
code { background: #f3f4f6; padding: 1px 4px; border-radius: 4px; }

.form { display: flex; flex-direction: column; gap: 14px; }
.field { display: flex; flex-direction: column; gap: 4px; }
label { font-weight: 600; font-size: 14px; }
input { width: 100%; padding: 10px 12px; font-size: 16px; border: 1px solid #d1d5db; border-radius: 8px;
}
input:focus { outline: 2px solid #4f46e5; outline-offset: 1px; border-color: transparent; }
input[aria-invalid="true"] { border-color: #dc2626; }
.field-error { color: #dc2626; font-size: 14px; }

.password-row { display: flex; gap: 6px; }
.toggle { padding: 0 14px; border: 1px solid #d1d5db; border-radius: 8px; background: white; color:
#111827; cursor: pointer; }

.summary { border: 2px solid #dc2626; border-radius: 8px; padding: 10px 14px; background: #fef2f2; }
.summary p { font-weight: 700; margin-bottom: 4px; }
.summary ul { padding-left: 18px; }
.summary a { color: #b91c1c; }

button[type="submit"] { padding: 12px; border: none; border-radius: 8px; background: #4f46e5; color:
white; font-size: 16px; cursor: pointer; }
button:disabled { opacity: 0.6; cursor: not-allowed; }
button:focus-visible, a:focus-visible { outline: 3px solid #a5b4fc; outline-offset: 2px; }
.status { color: #15803d; min-height: 20px; }
```

src/index.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
```

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: mount focus, first-invalid focus in visual order, live re-validation after submit, server field errors with focus (inputs read-only, not disabled), clearing server errors on edit, toggle that doesn't submit, double-submit guard, success reset and refocus, error summary links, accessible error wiring, whitespace input.

Trade-offs to mention to the interviewer

- *Auto-focus on mount:* great on a page whose only job is this form (login, search); harmful on content pages (screen-reader users miss what's above, mobile keyboards pop up). Ask first.
- *Focus first invalid field vs focus the error summary:* the field is fastest to fix; the summary (GOV.UK pattern) gives an overview first. We show both but focus the field.
- *Live validation only after submit:* avoids shouting at users while they type; on-blur validation is a good alternative for long forms.
- *Ref map vs* `document.getElementById` *:* refs stay inside React and work with duplicate forms on a page; `getElementById` is simpler but global.
- `readOnly` *vs* `disabled` *while submitting:* read-only keeps focusability (needed for server-error focus); disabled is visually clearer and excluded from the form data.

What to add with more time

- On-blur validation and an async "email available?" check (debounced, abortable).
- React Hook Form: `setFocus` / `shouldFocusError` do this out of the box; Zod schema shared with the backend.
- Multi-step version: focus the step heading when moving between steps.
- Tests (RTL + `user-event`): `toHaveFocus()` on mount, after an invalid submit, after a server error and after success; the toggle doesn't call `onSubmit`.

13. Counter questions the interviewer may ask

Requirements and design

Q1. Is auto-focusing an input on page load always good?

No. It's good when the form is the page's purpose (login, search, a modal). On content pages it skips the content above for screen-reader users, can scroll the page, and pops up the keyboard on mobile. WCAG doesn't ban it, but use it deliberately.

Q2. On submit with errors, should focus go to the first invalid field or to an error summary?

The first invalid field is the fastest to fix and its error is announced via `aria-describedby`. A focused summary (GOV.UK style) gives an overview of all errors and links to each. Both are accepted patterns; pick

one and be consistent. We show the summary but focus the field.

Q3. Should Enter move to the next field?

Generally no: Enter in a single-line input submits the form, and users and assistive tech expect that. Moving to the next field on Enter is a custom behaviour (sometimes requested for data-entry apps); then make the last field submit and document it.

State design

Q4. Why refs for focus instead of state?

Focus is an imperative action on a DOM element. Storing "which field is focused" in state means syncing state and DOM, with extra renders and effects. Refs give direct access to the node, and you call `.focus()` in the handler where you know it's needed.

Q5. What's a callback ref and why use it for the ref map?

`ref={(el) => { fieldRefs.current[name] = el; }}` : React calls it with the element on mount (and `null` on unmount). One `useRef({})` then holds any number of inputs keyed by name, instead of one `useRef` per field.

Q6. Why derive client errors but store server errors?

Client errors are a pure function of `values`, so storing them only risks staleness. Server errors come from an async response the client can't recompute, so they must be stored, and cleared when the related field changes.

Hard parts

Q7. Why does a button inside a form submit it?

The default `type` of `<button>` is `submit`. Any button without `type="button"` inside a `<form>` submits it on click or Enter. Always set `type` explicitly.

Q8. Why does `.focus()` sometimes "not work"?

The element is disabled, hidden (`display: none` / `visibility: hidden`), not rendered yet, or not focusable (a `div` without `tabIndex`). Also, focusing during the same render before the element exists does nothing; focus in an effect or after the state update that renders it (or use `flushSync`).

Q9. How do you focus a field that appears only after a state change (e.g. "Add another email")?

Set state to add the field, then focus it once it's rendered: an effect watching the fields list, a callback ref that focuses when `el` arrives, or `flushSync(() => setFields(...))` followed by `ref.focus()` .

Q10. How do you link an error message to its input for screen readers?

Give the message an `id` and set `aria-describedby="that-id"` on the input, plus `aria-invalid="true"` . When the input gets focus, the screen reader reads the label, the state and the error. Don't rely on red colour alone.

Q11. What's wrong with `alert("form submitted successfully")`?

It blocks the main thread and all interaction, can't be styled, looks different per browser, and can be suppressed. Use an inline status message (`role="status"`) or a toast, and keep focus somewhere sensible.

Implementation steps

Q12. How would you test the focus behaviour?

React Testing Library + `user-event`: after render, `expect(screen.getByLabelText("Full name")).toHaveFocus()`; type the name, `user.click(submit)` → `expect(emailInput).toHaveFocus()`; mock `registerUser` to reject with `field: "email"` → email has focus; resolve → name has focus and the fields are empty; click "Show" → submit handler not called.

Q13. How would React Hook Form do this?

`useForm({ shouldFocusError: true })` focuses the first field with an error on submit automatically (registration order), and `setFocus("email")` focuses a field imperatively (e.g. after a server error, together with `setError("email", { message })`).

Q14. How do you handle focus in a multi-step form?

On "Next", move focus to the new step's heading (`tabIndex={-1}`) so screen readers announce where they are, or to its first field. On "Back", restore focus to where the user was. Validate per step and focus the first invalid field of that step.

Performance and scale

Q15. A form has 60 fields. Any performance concerns?

Controlled inputs re-render the whole form on every keystroke; validating all 60 fields each render adds up. Use uncontrolled inputs (React Hook Form), split into memoised field components, validate only the changed field (+ dependents) live, and full validation on submit.

Q16. How would you validate "email already taken" before submit?

On blur (or debounced while typing), call a check endpoint with `AbortController` to cancel stale requests, show a "checking…" state, and still re-check on submit, because two people can register the same email at the same time.

Q17. Should validation rules live on the client or the server?

Both. Client rules are for fast feedback (UX); server rules are the real gate (security, data integrity), since the client can be bypassed. Share them with a schema library (Zod, Yup) when the stack allows.

Trade-offs and "why not X?"

Q18. Why not use the `autoFocus` prop?

React's `autoFocus` works (React calls `.focus()` on mount), and it's fine for the mount case. But it can't express "focus the first invalid field" or "refocus after success", and it fires on every mount, which matters when forms remount (routes, modals). A ref gives one consistent mechanism for all focus moves.

Q19. Why not native HTML validation (`required`, `type="email"`, `:invalid`)?

Native validation also focuses the first invalid field automatically. But its bubbles can't be styled, wording varies per browser, cross-field rules (confirm password) need JS anyway, and it validates only on submit. We keep `type="email"` and `autoComplete` for keyboards/autofill and use `noValidate` for our own accessible messages.

Q20. Why `readOnly` instead of `disabled` during submit?

Disabled inputs can't receive focus, so focusing the email field after a server error silently fails. Read-only inputs stay focusable and readable. Disabled is fine when nothing needs focus during or right after the request.

Q21. Why not move focus to the first field *while typing*, when it becomes invalid?

Never move focus while the user is typing: it's disorienting, steals keystrokes, and breaks WCAG 3.2.2 ("On Input": no unexpected context change). Move focus only in response to a deliberate action (submit, a link click, a step change).